Лабораторна робота №1

Функціональне програмування

Інформаційна система «Інформаційні ресурси кафедри»

Левківський Євгеній Сергійович

ІПС-41

1. Мета роботи

Розробити програму на Haskell з використанням класів типів та їхніх екземплярів. Спроєктувати реляційну базу MySQL, забезпечити введення, коригування й пошук інформації та побудову статистики використання ресурсів.

2. Постановка завдання

Обрано перше завдання: «Інформаційні ресурси кафедри». Для кожного ресурсу потрібно зберігати назву, авторів, анотацію, вид, призначення, дату відкриття, термін та умови використання, мережеві адреси, користувачів і статистику. Кількість таблиць має бути більшою за п’ять.

Реалізовано консольний застосунок. Код розміщено в lab1/src/ Department.hs та lab1/app/Main.hs, схему й початкові дані — в lab1/sql. Haskell виконує валідацію, формує запити й обробляє результати. Для передачі запитів справжньому серверу використано офіційний клієнт mysql через stdin/stdout.

3. Структура бази даних

База department_resources використовує utf8mb4. Схема містить 11 таблиць; представлення resource_statistics до їх кількості не включене.

Таблиця	Призначення
authors	Автори ресурсів
resource_types	Види ресурсів
purposes	Призначення ресурсів
usage_terms	Назви й тексти умов використання
resources	Назва, анотація, вид, умови, початок і кінець використання
resource_authors	Зв’язок ресурсів та авторів багато-до-багатьох
resource_purposes	Зв’язок ресурсів та призначень багато-до-багатьох
resource_urls	Мережеві адреси та шляхи до матеріалів
users	Користувач, група, унікальний логін
access_grants	Доступ користувача до ресурсу
usage_events	Відвідування, час і тривалість використання

Довідники відокремлено від ресурсів, що зменшує дублювання. Зовнішні ключі зберігають цілісність зв’язків. Подія usage_events посилається на складений ключ access_grants, тому використання без наданого доступу не приймається БД. Обмеження CHECK перевіряє, що кінець використання не передує початку. Значення NULL означає безстрокове використання.

Статистика містить число відвідувань, кількість різних користувачів та сумарну тривалість. LEFT JOIN дозволяє показувати також ресурси без відвідувань.

4. Типи, класи та екземпляри Haskell

Resource моделює ресурс, отриманий з БД; ResourceDraft містить поля для введення; MySQL описує параметри з’єднання. Логіка предметної області відділена від операцій введення-виведення.

```
class SqlValue a where
  sqlValue :: a -> String

class Validatable a where
  validate :: a -> Either String ()

class ResourceStore store where
  listResources :: store -> IO [Resource]
  searchResources :: store -> Text -> IO [Resource]
  insertResource :: store -> ResourceDraft -> IO Int
  updateResource :: store -> Int -> ResourceDraft -> IO Bool
  deleteResource :: store -> Int -> IO Bool
```

SqlValue має екземпляри для Text, Int, Day і Maybe a. Екземпляр Validatable ResourceDraft перевіряє назву, анотацію, ідентифікатори та календарні дати. Екземпляр ResourceStore MySQL реалізує CRUD і пошук. Це скорочений запис інтерфейсів; повні визначення та імпорти містяться у вихідних файлах.

Текстові параметри перетворюються на UTF-8 hex-літерали CONVERT(X'...' USING utf8mb4). Тому лапки, крапки з комою та SQL-оператори у введеному тексті не стають кодом запиту. У команді edit назви таблиць і стовпців перевіряються за явним списком дозволених значень.

5. Алгоритм роботи та запуск

Програма читає команду й конфігурацію MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_DATABASE та MYSQL_BIN. Пароль передається клієнту через MYSQL_PWD. Після перевірки параметрів формується SQL, клієнт виконує запит, а Haskell перетворює рядки результату на Resource або табличний вивід. Помилки MySQL повертаються з ненульовим кодом завершення.

```
node scripts/local.mjs lab1 list
node scripts/local.mjs lab1 search Haskell
node scripts/local.mjs lab1 catalog
node scripts/local.mjs lab1 details 1
node scripts/local.mjs lab1 stats
```

Команди add та update приймають назву, анотацію, вид, умови, дату початку й дату завершення; символ «-» позначає відсутність кінцевої дати. Додаткові команди створюють авторів, види, призначення, умови, користувачів, зв’язки й адреси, надають доступ і реєструють відвідування.

Команда edit коригує довідники, користувачів та адреси. Детальні приклади наведено в lab1/README.md.

6. Результати перевірки

Перевірку виконано на Windows x64, GHC 9.6.7 та MySQL Community Server 8.4.6. Використано справжній локальний сервер на 127.0.0.1:3307. Повний журнал наведено у verification/lab1.txt.

Початкові дані містять три ресурси. Для GHC User Guide статистика показує 2 відвідування, 1 користувача та 480 секунд; для MySQL Reference Manual — 1 відвідування й 240 секунд; третій ресурс має нуль відвідувань.

Перевірка	**Результат**
Перегляд, пошук, довідники та зв’язки	Виконано успішно
Створення та оновлення українського тексту	Дані збережено і знайдено
Лапки, табуляція й новий рядок у параметрах	Збережено як дані
Пошук рядка з SQL-ін’єкцією	Немає сторонніх результатів
Надання доступу й реєстрація 90 секунд	Статистика: 1 відвідування, 1 користувач, 90 секунд
Некоректна дата та відсутній зовнішній ключ	Операції відхилено
Невідома таблиця в edit	Відхилено до виконання SQL
Видалення тестового ресурсу	Ресурс і залежні записи видалено

7. Висновок

Реалізовано інформаційну систему з 11 пов’язаних таблиць MySQL. Використано класи SqlValue, Validatable, ResourceStore та відповідні instances. Забезпечено пошук, введення, коригування, зв’язки між ресурсами й довідниками та облік використання. Роботу з БД і перевірки даних підтверджено автоматизованими запусками.

8. Джерела

GHC User’s Guide: https://downloads.haskell.org/ghc/9.6.7/docs/users_guide/

MySQL 8.4 Reference Manual: https://dev.mysql.com/doc/refman/8.4/en/